Probability and Gaussian basics

Formula first; every symbol explained below. Natural logs unless stated.

1. Joint, marginal and Bayes

$$p(x, y) = p(x \mid y)p(y), \quad p(x) = \sum_y p(x \mid y)p(y)$$

$$p(y \mid x) = \frac{p(x \mid y)p(y)}{\sum_k p(x \mid k)p(k)}$$

x – One observation, such as an image or a word feature.
y – A class label or conditioning category.
p(x,y) – Joint probability of x and y occurring together.
p(x|y) – Likelihood of observation x within class y.
p(y) – Prior probability of class y before seeing x.
p(x) – Evidence: total probability of observing x.
p(y|x) – Posterior probability of class y after seeing x.
k in the denominator – Index over all possible classes.
Remember: Multiply likelihood by prior, then normalize across classes.

2. Expectations and variance

$$\mathbb{E}[f(X)] = \sum_x p(x)f(x), \quad \hat{\mathbb{E}}[f] = \frac{1}{N}\sum_{i=1}^{N} f(x_i)$$

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2, \quad \sigma = \sqrt{v}$$

$$\mathrm{Var}(aX + bE) = a^2\mathrm{Var}(X) + b^2\mathrm{Var}(E)$$

X – A random variable; x is one possible value of X.
f(X) – The quantity whose probability-weighted average we want.
E[...] – Expectation, or a probability-weighted average.
E with a hat – A finite-sample estimate of the expectation.
N – Number of observed or sampled values in the average.
i – Sample index; x_i is the i-th observation.
Var(X) – Expected squared spread around the mean of X.
E[X^2] – Expected square, not the square of the expectation.
v – Variance: squared spread.
σ (sigma) – Standard deviation: square root of variance.
a, b – Constant scaling coefficients applied to random variables.
E in aX+bE – A second random variable, distinct from expectation E[...].
Remember: The variance-addition formula assumes independence or zero covariance.

3. Gaussian density

$$N(z; \mu, v) = \frac{1}{\sqrt{2\pi v}} e^{-(z-\mu)^2/(2v)}$$

N(z;μ,v) – Gaussian density evaluated at z, with mean μ and variance v.
z – The value at which the density is evaluated.
μ (mu) – Mean: the center of the Gaussian distribution.
v > 0 – Variance: controls the Gaussian spread.
π (pi) – The constant approximately 3.14159.
e – Euler's number, approximately 2.71828.
exp(a) = e^a – Exponential function, inverse of natural logarithm.
Remember: A continuous density at one point is not its point probability.

Autoencoders, gradients and latent geometry

Formula first; every symbol explained below. Natural logs unless stated.

1. Encoder, decoder and a linear layer

$$z = f_\phi(x), \quad \hat{x} = g_\theta(z), \quad u = xW + b$$

x – Input observation or row vector of input features.
z – Latent code produced by the encoder.
f_phi – Encoder function with trainable parameters phi.
phi (φ) – Collection of encoder weights and biases.
g_theta – Decoder function with trainable parameters theta.
theta (θ) – Collection of decoder/model weights and biases.
x_hat – Reconstructed input produced by the decoder.
u – Output feature vector of the affine layer.
W – Trainable matrix that mixes input coordinates.
b – Trainable bias vector added after multiplication.

2. Reconstruction loss and anomaly threshold

$$SSE = \sum_{d=1}^{D} (\hat{x}_d - x_d)^2, \quad MSE = \frac{SSE}{D}$$

$$\frac{\partial MSE}{\partial \hat{x}_d} = \frac{2}{D}(\hat{x}_d - x_d), \quad \text{anomaly if } MSE > \tau$$

SSE – Sum of squared reconstruction errors.
MSE – Mean squared error: SSE divided by data dimension D.
D – Number of compared data coordinates; not the GAN discriminator.
d – Coordinate index in the reconstruction vector.
x_d, x_hat_d – True and reconstructed values of coordinate d.
∂MSE/∂x_hat_d – Sensitivity of MSE to that reconstructed coordinate.
τ (tau) – Error threshold above which the input is flagged.
Remember: With a strict > rule, an error equal to tau is not flagged.

3. Gradient descent and chain rule

$$\theta \leftarrow \theta - \eta \nabla_\theta L, \quad \frac{\partial L}{\partial w} = \sum_j \frac{\partial L}{\partial u_j} \frac{\partial u_j}{\partial w}$$

L – Scalar loss to minimize.
θ (theta) – Trainable parameter or parameter vector being updated.
η (eta) – Learning rate: size of the descent step.
grad_theta L – Vector of loss derivatives with respect to theta.
w – One trainable parameter whose derivative is needed.
u_j – Intermediate value on the j-th computational path.
∂L/∂w – Local rate of loss change when w changes.
sum over j – Add contributions from every path reaching w.
Remember: Multiply derivatives along a path; add across paths; subtract the gradient.

Distances, interpolation and VAE sampling

Formula first; every symbol explained below. Natural logs unless stated.

1. Retrieval distances

$$d(a,b) = \sqrt{\sum_j (a_j - b_j)^2}, \quad \cos(a,b) = \frac{a^\top b}{\|a\|\|b\|}$$

a, b – Feature or latent vectors being compared.
j – Coordinate index within a vector.
d(a,b) – Euclidean distance: straight-line separation.
||a|| – Euclidean vector length, square root of sum of squares.
a^T b – Dot product: multiply matching entries and sum.
T superscript – Transpose: interchange row and column orientation.
cos(a,b) – Cosine similarity: normalized dot product.
Remember: Cosine similarity is undefined if either vector is zero.

2. Latent interpolation and attribute editing

$$z(\lambda) = (1-\lambda)z_A + \lambda z_B, \quad 0 \le \lambda \le 1$$

$$\Delta z = z_{attribute} - z_{base}, \quad z_{edited} = z_{other} + \Delta z$$

z_A, z_B – Starting and ending latent codes.
λ (lambda) – Interpolation fraction from 0 to 1.
z(λ) – Intermediate latent code to decode.
z_attribute – Code representing the attribute-bearing example.
z_base – Reference code without that attribute.
Δz (delta z) – Estimated latent direction for the attribute.
z_other – Another code to which the direction is applied.
z_edited – Resulting code after adding the attribute direction.
Remember: A nonlinear decoder need not produce pixel averages or guaranteed edits.

3. VAE encoder distribution and sample

$$q_\phi(z \mid x) = N(\mu, diag(\sigma^2)), \quad p(z) = N(0, I)$$

$$\ell = \ln \sigma^2, \quad v = e^\ell, \quad \sigma = e^{\ell/2} = \sqrt{v}$$

$$\varepsilon \sim N(0, I), \quad z = \mu + \sigma \odot \varepsilon$$

q_phi(z|x) – Encoder's approximate latent distribution for input x.
p(z) – Prior distribution used when generating new samples.
μ – Encoder-predicted mean vector.
σ^2 or v – Encoder-predicted variance vector.
diag(σ^2) – Covariance matrix with those variances on its diagonal.
I – Identity matrix: unit variance and zero cross-covariance here.
ell (ell) – Log variance, ln(σ^2); not standard deviation.
ln – Natural logarithm, with base e.
σ – Standard deviation, exp(ell/2), used in sampling.
ε (epsilon) – Standard Gaussian noise drawn independently of parameters.
circle-dot ((elementwise)) – Multiply matching vector coordinates elementwise.
z – Sampled latent vector passed to the decoder.
~ symbol – Means drawn from the distribution on its right.
Remember: Reconstruct using q_phi(z|x); generate new data using the prior p(z).

KL divergence and Gaussian regularization

Formula first; every symbol explained below. Natural logs unless stated.

1. Discrete and expectation forms of KL

$$D_{KL}(P\|Q) = \sum_j P_j \ln \frac{P_j}{Q_j} \geq 0$$

$$D_{KL}(q\|p) = \mathbb{E}_q[\ln q(z) - \ln p(z)]$$

D_KL(P||Q) - Directional mismatch from distribution P to Q.
P_j - Probability of outcome j in the first distribution.
Q_j - Probability of outcome j in the comparison distribution.
j - Index over the discrete outcomes.
P_j / Q_j - Probability ratio whose natural log is measured.
sum over j - Add the probability-weighted contributions.
q(z), p(z) - First and comparison probability densities for latent z.
E_q - Average under the FIRST distribution q.
|| inside KL - Separates the two distributions; not a vector norm.
Remember: KL is nonnegative and asymmetric. Zero P terms contribute zero.

2. Diagonal Gaussian KL to N(0,I)

$$K = \frac{1}{2}\sum_j \left(\mu_j^2 + v_j - 1 - \ln v_j\right)$$

$$K = -\frac{1}{2}\sum_j \left(1 + \ell_j - \mu_j^2 - e^{\ell_j}\right)$$

K - Total KL penalty for one encoded observation.
j - Latent-coordinate index, summed across the code.
μ_j - Mean of latent coordinate j.
v_j = σ_j^2 - Variance of latent coordinate j.
ell_j - Natural log of v_j.
exp(ell_j) - Recovers the variance v_j.
μ_j^2 - Penalty for displacement of the latent mean from zero.
v_j - 1 - ln(v_j) - Penalty for variance differing from one.
Remember: These formulas assume a diagonal encoder Gaussian and standard-normal prior.

3. KL gradients

$$\frac{\partial K}{\partial \mu_j} = \mu_j, \quad \frac{\partial K}{\partial \ell_j} = \frac{1}{2}(e^{\ell_j} - 1)$$

∂K/∂μ_j - KL derivative with respect to latent mean j.
∂K/∂ell_j - KL derivative with respect to latent log variance j.
μ_j - Mean value that also equals its KL derivative.
exp(ell_j)-1 - Variance minus the prior variance of one.
Remember: Check: mu=0 and variance=1 gives both KL and its gradients zero.

VAE objectives and reparameterization

Formula first; every symbol explained below. Natural logs unless stated.

1. ELBO and weighted VAE objective

$$ELBO = \mathbb{E}_{q_\phi}[\ln p_\theta(x \mid z)] - K$$

$$L_{VAE} = \mathbb{E}_{q_\phi}[-\ln p_\theta(x \mid z)] + \beta_{KL} K$$

$$-\ln p_\theta(x \mid z) = \frac{\|x - \hat{x}\|^2}{2s^2} + constant$$

ELBO - Evidence lower bound on the data log-likelihood.
q_phi(z|x) - Encoder distribution used for the latent expectation.
p_theta(x|z) - Decoder likelihood of the input given a latent code.
ln p_theta(x|z) - Reconstruction log-likelihood to maximize.
-ln p_theta(x|z) - Reconstruction negative log-likelihood to minimize.
K - KL from the encoder distribution to the latent prior.
L_VAE - Reconstruction penalty plus weighted KL penalty.
β_KL - KL coefficient; one gives the usual negative ELBO.
x, x_hat - True and reconstructed observation vectors.
s^2 - Fixed observation variance of a Gaussian decoder.
constant - Term independent of the reconstructed mean for fixed s^2.
Remember: The Gaussian-likelihood scale matters when relating NLL to MSE.

2. Explicit pixel/latent/batch reductions

$$L_{batch} = \frac{1}{B} \sum_{i=1}^{B} \left[\frac{1}{D} \sum_{d} (\hat{x}_{id} - x_{id})^2 + \beta_{KL} K_i \right]$$

L_batch - Mean objective across the minibatch.
B - Number of observations in the minibatch.
i - Observation index within the minibatch.
D - Number of data/pixel coordinates per observation.
d - Coordinate index in that observation.
x_id, x_hat_id - True and predicted coordinate d of observation i.
K_i - KL summed over latent coordinates for observation i.
β_KL - Weight of the KL term relative to reconstruction.
Remember: Averaging pixels but summing latent KL is a declared convention, not a universal one.

3. Reparameterization derivatives

$$\frac{\partial z_j}{\partial \mu_j} = 1, \quad \frac{\partial z_j}{\partial \sigma_j} = \varepsilon_j, \quad \frac{\partial z_j}{\partial \ell_j} = \frac{1}{2} \sigma_j \varepsilon_j$$

z_j - Sampled value of latent coordinate j.
μ_j - Mean of that latent coordinate.
σ_j - Standard deviation of that latent coordinate.
ε_j - Sampled standard-normal noise, held fixed during backprop.
ell_j - Log variance used to compute σ_j.
∂z_j/∂μ_j - Unit sensitivity to the latent mean.
∂z_j/∂σ_j - Sensitivity to standard deviation equals ε_j.
∂z_j/∂ell_j - Sensitivity to log variance equals half σ_j ε_j.
Remember: Add reconstruction and KL gradient paths, then apply weights and reductions.

GAN objectives and discriminator gradients

Formula first; every symbol explained below. Natural logs unless stated.

1. The adversarial game

$$V(D,G) = \mathbb{E}_{x\sim p_{data}} \ln D(x) + \mathbb{E}_z \ln(1 - D(G(z)))$$

$$\min_G \max_D V(D,G), \quad L_D = -V(D,G)$$

$$L_{G,sat} = \mathbb{E}_z \ln(1 - D(G(z)))$$

$$L_{G,NS} = -\mathbb{E}_z \ln D(G(z))$$

G – Generator mapping a latent noise vector to a fake observation.
D – Discriminator scoring how real an observation appears.
x – A real observation sampled from the training distribution.
p_data – Real-data distribution.
z – Random generator input drawn from the latent prior.
G(z) – Generated observation.
D(x), D(G(z)) – Discriminator real-probability scores on real/fake data.
V(D,G) – Value function maximized by D and minimized by G.
L_D – Discriminator minimization loss: negative value function.
L_G,sat – Saturating/minimax generator minimization loss.
L_G,NS – Non-saturating generator minimization loss.
E_x, E_z – Averages over real data and latent noise, respectively.
Remember: Separate real/fake means differ in scale from a combined average.

2. Sigmoid, BCE and its logit derivative

$$q = \sigma(a) = \frac{1}{1+e^{-a}}, \quad \frac{dq}{da} = q(1-q)$$

$$BCE(y,q) = -y \ln q - (1-y)\ln(1-q)$$

$$\frac{\partial BCE}{\partial a} = q - y$$

a – Discriminator logit before the sigmoid function.
σ(a) – Sigmoid function; here sigma does not mean standard deviation.
q – Predicted real-probability score, sigmoid(a).
y – Binary target: 1 for real, 0 for fake in a D update.
BCE(y,q) – Binary cross-entropy between target and prediction.
dq/da – Sigmoid derivative, q times one minus q.
∂BCE/∂a – Per-example logit derivative, prediction minus target.
Remember: During a non-saturating G update, fakes use target y=1.

GAN feedback, conditioning and StackGAN

Formula first; every symbol explained below. Natural logs unless stated.

1. Generator logit and parameter gradients

$$\frac{\partial L_{G,sat}}{\partial a} = -q, \quad \frac{\partial L_{G,NS}}{\partial a} = q - 1$$
$$\nabla_{\theta_G} L_{G,NS} = \mathbb{E}_z[(q-1)\nabla_{\theta_G} a]$$

a – Discriminator logit for the generated observation.
q – Sigmoid(a), the fake observation's real score.
∂L_G,sat/∂a – Per-example saturating logit gradient, −q.
∂L_G,NS/∂a – Per-example non-saturating logit gradient, q−1.
θ_G – Generator parameters being updated.
grad_thetaG a – Remaining derivative through D and G to generator weights.
E_z – Average over latent inputs used in the G objective.
Remember: Near q=0, gradients are near 0 versus −1. Frozen D weights still pass input gradients.

2. Ideal discriminator (balanced real/fake priors)

$$D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}$$

D*(x) – Optimal discriminator score for fixed generator distribution.
p_data(x) – Density of real data at x.
p_G(x) – Density induced by the generator at x.
Remember: If p_G=p_data, optimal D=0.5; a score of 0.5 alone does not prove matching.*

3. Conditional GAN objective

$$V_c = \mathbb{E}_{x,y} \ln D(x,y) + \mathbb{E}_{z,y} \ln(1 - D(G(z,y),y))$$

V_c – Conditional adversarial value function.
y – Requested condition: label, text representation or image.
D(x,y) – Discriminator score for an observation-condition pair.
G(z,y) – Generator output using both noise and condition.
E_(x,y) – Average over matching real observation-condition pairs.
E_(z,y) – Average over generated noise-condition pairs.
Remember: Both networks receive y; the discriminator judges realism AND matching.

4. Conditioning augmentation

$$\hat{c} = \mu(c) + \sigma(c) \odot \varepsilon, \quad \varepsilon \sim N(0, I)$$

c – Text or other condition representation.
μ(c) – Mean predicted from that condition.
σ(c) – Standard deviation predicted from that condition.
ε – Standard-normal random conditioning noise.
c_hat – Sampled, augmented conditioning vector.
circle-dot ((elementwise)) – Elementwise multiplication of scale and noise.
Remember: Use the same variance/log-variance conversion and Gaussian KL as a VAE.

Convolution sizes and parameter counts

Formula first; every symbol explained below. Natural logs unless stated.

1. Convolution and transposed convolution

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - k}{s} \right\rfloor + 1$$

$$H_{out}^{transpose} = (H_{in} - 1)s - 2p + k$$

H_in – Input spatial height.
H_out – Output spatial height.
k – Kernel size in the height direction.
p – Padding on each side in this direction.
s – Stride: displacement between neighboring kernel positions.
floor brackets – Round the enclosed value down to an integer.
H_out,transpose – Output height of the transposed convolution.
Remember: These formulas assume dilation=1; the transpose formula also assumes output padding=0.

2. Shared convolution weights

$$\#conv\ params = k_h k_w C_{in} C_{out} + C_{out}$$

k_h, k_w – Kernel height and width.
C_in – Number of input channels.
C_out – Number of output channels/filters.
k_h k_w C_in C_out – Number of shared kernel weights.
final C_out term – One optional bias per output channel.
#conv params – Total trainable convolution parameters.
Remember: Spatial positions do not multiply the count of shared filter weights.

3. Supporting shape arithmetic

H, W, C – Image height, width and number of channels.
H x W x C – Number of scalar pixel-channel values in an image.
dim(z)+dim(y) – Width of a concatenated noise-condition input.
sum K_j – Width when concatenating one-hot categorical fields.
product K_j – Number of possible combinations of those fields.
K_j – Number of categories in categorical field j.
resolution ratio – At fixed channels, pixel count scales with height x width.
Remember: 64x64 to 256x256 grows each side by 4, so total values grow by 16.

Entropy and Inception Score

Formula first; every symbol explained below. Natural logs unless stated.

1. Entropy and the marginal class distribution

$$H(P) = -\sum_c P_c \ln P_c, \quad m_c = \frac{1}{N}\sum_{i=1}^{N} P_{ic}$$

H(P) – Uncertainty of a probability vector P.
P_c – Probability of class c in vector P.
c – Class index.
N – Number of generated images.
i – Generated-image index.
P_ic – Classifier probability of class c for generated image i.
m_c – Marginal class probability: mean of column c.
ln – Natural logarithm; entropy is measured in nats.
Remember: Want low uncertainty per image, but varied predicted classes across the set.

2. KL rows, mean and exponentiation

$$K_i = \sum_c P_{ic} \ln \frac{P_{ic}}{m_c}, \quad IS = \exp\left(\frac{1}{N}\sum_i K_i\right)$$

K_i – KL from image i's class vector to the marginal m.
P_ic / m_c – Ratio of image-specific to average class probability.
sum over c – Add probability-weighted contributions across classes.
(1/N) sum_i K_i – Mean KL across generated images.
IS – Inception Score: exponential of that mean KL.
exp – Exponential function with base e.
Remember: Do not additionally divide each row KL by the number of classes.

3. Entropy identity and score bounds

$$\ln IS = H(m) - \frac{1}{N}\sum_i H(P_i) = I(X;Y)$$

$$1 \le IS \le C, \quad P_i = m \ \forall i \implies IS = 1$$

H(m) – Entropy of the marginal predicted class distribution.
H(P_i) – Entropy of the classifier vector for image i.
I(X;Y) – Mutual information between generated image and classifier label.
X – Randomly selected generated image in this identity.
Y – Classifier label sampled from its prediction vector.
C – Number of classifier classes; an upper bound on IS.
P_i=m for all i – Every image has the same prediction distribution.
Remember: Identical predictions give IS=1, even if confident; higher scores do not prove novelty.

FID, precision, recall and PSNR

Formula first; every symbol explained below. Natural logs unless stated.

1. Feature mean and covariance

$$\hat{\mu} = \frac{1}{N}\sum_i f_i, \quad \hat{\Sigma} = \frac{1}{N-1}\sum_i (f_i - \hat{\mu})(f_i - \hat{\mu})^T$$

f_i – Feature vector extracted from image i.
N – Number of feature samples; covariance formula needs N>1.
μ_hat – Sample mean of the feature vectors.
Σ_hat (Sigma) – Sample covariance matrix of the feature vectors.
N-1 denominator – Unbiased sample-covariance convention.
T superscript – Transpose, forming an outer product of deviations.

2. FID and its diagonal shortcut

$$FID = \|\mu_r - \mu_g\|^2 + Tr(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2})$$

$$FID_{diagonal} = \sum_j (\mu_{rj} - \mu_{gj})^2 + \sum_j (\sqrt{v_{rj}} - \sqrt{v_{gj}})^2$$

FID – Gaussian feature-moment distance; lower is preferred.
μ_r, μ_g – Mean feature vectors for real and generated images.
Σ_r, Σ_g – Covariance matrices for real and generated features.
||μ_r-μ_g||^2 – Squared Euclidean separation between feature means.
Tr(...) – Trace: sum of the matrix's diagonal entries.
(Σ_r Σ_g)^(1/2) – Matrix square root of the covariance product.
j – Feature-coordinate index in the diagonal shortcut.
v_rj, v_gj – Real/generated feature variances in coordinate j.
sqrt(v_rj), sqrt(v_gj) – Corresponding standard deviations.
Remember: FID=0 means matching Gaussian moments, not proven equality or no copying.

FID, precision, recall and PSNR (continued)

Formula first; every symbol explained below. Natural logs unless stated.

3. kNN balls and support membership

$$r_k(u_i; S) = \|u_i - NN_k(u_i; \{u_j : j \neq i\})\|_2$$

$$F(a; S) = \mathbf{1}\{\exists u \in S : \ \|a - u\|_2 \leq r_k(u; S)\}$$

S – Reference collection of feature vectors.
u_i – Center of the i-th reference ball.
k – Which nearest OTHER reference neighbor sets the radius.
NN_k – k-th nearest neighbor in the supplied candidate collection.
j != i – Exclude the sample itself by index.
r_k(u_i;S) – Radius of the ball centered at reference point u_i.
a – Query feature vector being tested against the balls.
F(a;S) – Membership indicator: 1 if any reference ball covers a.
exists symbol – At least one reference center must satisfy the test.
||a-u||_2 – Euclidean distance from query a to center u.
Remember: Test the union of balls, including the boundary; nearest center alone can fail.

4. Two directions and an image metric

$$\text{Precision} = \frac{1}{|G|} \sum_{g \in G} F(g; R)$$

$$\text{Recall} = \frac{1}{|R|} \sum_{r \in R} F(r; G)$$

$$\text{PSNR} = 10 \log_{10} \frac{\text{MAX}^2}{\text{MSE}}$$

R, G – Real and generated feature collections, respectively.
|R|, |G| – Counts of real and generated samples.
g, r – One generated query or one real query.
Precision – Fraction of generated points inside real balls.
Recall – Fraction of real points inside generated balls.
PSNR – Peak signal-to-noise ratio, usually reported in dB.
MAX – Stated maximum clean pixel value, such as 1 or 255.
MSE – Mean squared error in the stated image value scale.
log_10 – Base-ten logarithm, unlike ln used in the other losses.
Remember: For PSNR, MSE=0 gives infinity. State per-image versus pooled averaging.

Diffusion: forward process and aggregate noise

Formula first; every symbol explained below. Natural logs unless stated.

1. Schedule symbols and one-step noising

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s, \quad \bar{\alpha}_0 = 1$$

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{\beta_t}\varepsilon_t, \quad \varepsilon_t \sim N(0, I)$$

$$q(x_t \mid x_{t-1}) = N(\sqrt{\alpha_t} x_{t-1}, \beta_t I)$$

t - Current diffusion timestep.
β_t (beta) - Variance contributed by fresh noise at step t.
α_t (alpha) - One-step signal variance share: 1-beta_t.
alpha_bar_t - Product of all alphas from step 1 through t.
product symbol (Π) - Multiply the indexed factors together.
x_(t-1), x_t - Noisy observations before and after this forward step.
ε_t - Fresh standard-normal step noise, independent of earlier draws.
q(x_t|x_(t-1)) - Fixed forward transition distribution.
I - Identity covariance matrix for standard Gaussian noise.
Remember: Beta is a variance. Square roots are amplitude coefficients.

2. Direct noising, variance and the aggregate draw

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon$$

$$q(x_t \mid x_0) = N(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I)$$

$$\mathrm{Var}(x_t) = \alpha_t \mathrm{Var}(x_{t-1}) + \beta_t$$

$$\mathrm{Var}(x_t \mid x_0) = (1 - \bar{\alpha}_t) I$$

$$\alpha_2\beta_1 + \beta_2 = 1 - \alpha_1\alpha_2$$

$$\varepsilon = \frac{x_t - \sqrt{\bar{\alpha}_t} x_0}{\sqrt{1 - \bar{\alpha}_t}}$$

$$\varepsilon = \frac{\sum_{s=1}^{t} \sqrt{\beta_s} \left(\prod_{j=s+1}^{t} \sqrt{\alpha_j} \right) \varepsilon_s}{\sqrt{1 - \bar{\alpha}_t}}$$

x_0 - Original clean observation.
ε without a step index - Aggregate standard-normal noise in the one-shot formula.
sqrt(alpha_bar_t) - Cumulative clean-signal amplitude coefficient.
sqrt(1-alpha_bar_t) - Aggregate-noise amplitude coefficient.
q(x_t|x_0) - Distribution of timestep-t noise conditioned on clean x_0.
Var(x_t) - Marginal variance across both data and corruption noise.
Var(x_t|x_0) - Conditional covariance for a fixed clean observation.
α_2 β_1+β_2 - Variance of the combined independent two-step noise.
ε - Aggregate draw corresponding to the supplied noisy sample.
s - Index of an incremental noise contribution.
β_s, ε_s - Noise variance and fresh draw introduced at step s.
j - Index of later steps that attenuate that contribution.
product from s+1 to t - Remaining signal multipliers after noise was added.
empty product - Equals 1, including the final step's noise contribution.
Remember: Conditional variance is 1-alpha_bar_t. A fresh aggregate draw matches the distribution, not necessarily the sample.

Diffusion: schedules and true posterior

Formula first; every symbol explained below. Natural logs unless stated.

1. Linear and normalized cosine schedules

$$\beta_t = \beta_{start} + \frac{t-1}{T-1}(\beta_{end} - \beta_{start})$$

$$f(t) = \cos^2\left(\frac{t/T + s}{1+s}\frac{\pi}{2}\right), \quad s = 0.008$$

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad \beta_t = \min\left(1 - \frac{f(t)}{f(t-1)}, 0.999\right)$$

t - Step index, from 1 through T for the linear schedule.
T - Total number of diffusion steps.
β_start, β_end - First and last noise variances of the linear schedule.
f(t) - Unnormalized cosine-squared cumulative signal curve.
s in f(t) - Small cosine offset; the lecture uses 0.008.
f(0) - Normalizer ensuring alpha_bar_0 equals 1.
π/2 - Half-pi angle; use radians.
β_t - Derived noise variance for the cosine schedule.
min(...,0.999) - Cap that prevents a single beta from reaching 1.
Remember: After capping beta, recompute realized cumulative alphas from their product.

2. Posterior when the clean image is known

$$q(x_{t-1} \mid x_t, x_0) = N(\tilde{\mu}_t, \tilde{\beta}_t I)$$

$$\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$$

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t$$

q(x_(t-1)|x_t,x_0) - True reverse posterior conditioned on clean x_0.
x_t, x_(t-1) - Current noisy observation and previous less-noisy state.
x_0 - Clean observation, known in training but unavailable in generation.
mu_tilde_t - Mean of the true Gaussian posterior.
beta_tilde_t - Variance of the true Gaussian posterior.
α_t, β_t - One-step signal and added-noise variance shares.
alpha_bar_(t-1) - Cumulative signal share before the current step.
alpha_bar_t - Cumulative signal share through the current step.
I - Identity matrix giving isotropic posterior covariance.
Remember: Use this Gaussian posterior for t>1; the final reconstruction term is handled separately.

Diffusion: reverse sampling and training

Formula first; every symbol explained below. Natural logs unless stated.

1. Clean estimate and reverse mean

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right)$$

x_0_hat – Estimated clean observation implied by predicted noise.
x_t – Current noisy sample available to the network.
ε_theta(x_t,t) – Neural network's prediction of aggregate noise.
θ (theta) – Denoiser parameters learned during training.
μ_theta(x_t,t) – Predicted mean of the next reverse transition.
α_t, β_t – Signal/noise schedule values at the current step.
alpha_bar_t – Cumulative signal variance share.
Remember: Derive the reverse mean by replacing x_0 with x_0_hat in the true posterior mean.

2. A complete stochastic reverse transition

$$x_T \sim N(0, I), \quad x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z$$

$$z \sim N(0, I)\ (t > 1), \quad z = 0\ (t = 1)$$

x_T – Initial Gaussian sample from which generation starts.
T – Final/maximum timestep; sampling runs down to 1.
x_(t-1) – Sample produced by the current reverse transition.
σ_t – Standard deviation of the chosen reverse sampling noise.
z – Fresh standard-normal reverse-sampling draw.
μ_theta – Center of the reverse sampling distribution.
z=0 at t=1 – Omit added noise in the final DDPM step.
Remember: State the variance choice, e.g. sigma_t squared=beta_t or beta_tilde_t.

3. Noise-prediction training objective

$$t \sim \text{Uniform}\{1, \ldots, T\}, \quad L_{simple} = \mathbb{E}_{t, x_0, \varepsilon}\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$$

$$L_{MSE} = \frac{1}{D}\sum_j (\hat{\varepsilon}_j - \varepsilon_j)^2, \quad \frac{\partial L_{MSE}}{\partial \hat{\varepsilon}_j} = \frac{2}{D}(\hat{\varepsilon}_j - \varepsilon_j)$$

t~Uniform{1,...,T} – Sample a training timestep uniformly.
x_0 – Clean training example used to manufacture x_t.
ε – True aggregate noise used in that manufacture.
ε_theta or ε_hat – Predicted noise from input pair (x_t,t).
L_simple – Expected squared noise-prediction error.
L_MSE – Coordinate-mean version of squared noise error.
D – Number of noise coordinates being averaged.
j – Noise-coordinate index.
∂L_MSE/∂ε_hat_j – Output gradient passed into the denoiser.
Remember: Training uses random t; generation uses descending t. No full reverse chain per training image.

Diffusion: the variational bound

Formula first; every symbol explained below. Natural logs unless stated.

1. Bound and its three kinds of terms

$$L_{VLB} = \mathbb{E}_q\left[L_T + \sum_{t=2}^{T} L_{t-1} + L_0\right]$$

$$L_T = D_{KL}(q(x_T \mid x_0) \| p(x_T)), \quad L_0 = -\ln p_\theta(x_0 \mid x_1)$$

$$L_{t-1} = D_{KL}(q(x_{t-1} \mid x_t, x_0) \| p_\theta(x_{t-1} \mid x_t))$$

L_VLB - Variational upper bound on expected negative data log-likelihood.
E_q - Average under the fixed forward corruption process.
L_T - Endpoint KL comparing the terminal forward noise to the prior.
sum from t=2 to T - Add the intermediate reverse-posterior KL terms.
L_(t-1) - KL between true and learned reverse transitions at that step.
L_0 - Final-step reconstruction negative log-likelihood.
q(x_T|x_0) - Terminal forward distribution for a clean example.
p(x_T) - Chosen terminal prior, usually N(0,I).
q(x_(t-1)|x_t,x_0) - True reverse posterior used as the training reference.
p_theta(x_(t-1)|x_t) - Learned reverse transition distribution.
p_theta(x_0|x_1) - Learned final reconstruction likelihood.
Remember: The bound includes endpoint, intermediate-posterior and reconstruction terms.

2. Gaussian KL becomes squared mean error

$$D_{KL}(N(m_q, s^2 I) \| N(m_p, s^2 I)) = \frac{\|m_q - m_p\|^2}{2s^2}$$

m_q, m_p - Mean vectors of the two Gaussian distributions.
s^2 - Common isotropic variance of BOTH Gaussian distributions.
I - Identity matrix giving the covariance s^2 I.
||m_q-m_p||^2 - Squared Euclidean separation of the mean vectors.
2s^2 - Variance-dependent scaling of the mean mismatch.
Remember: This exact reduction requires equal covariances.

3. Weighted noise error and the simplification

$$L_{t-1} = C_t + w_t \|\varepsilon - \varepsilon_\theta\|^2, \quad w_t = \frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)}$$

C_t - Term independent of denoiser parameters for fixed variances.
w_t - Schedule-dependent weight on timestep-t noise error.
ε, ε_theta - True aggregate and predicted aggregate noise.
β_t - Forward added-noise variance.
σ_t^2 - Fixed reverse-model variance, not standard deviation.
α_t - One-step retained signal variance share.
alpha_bar_t - Cumulative retained signal variance share.
Remember: Dropping w_t changes weighting across timesteps; it is not identical optimization.

Transformers: embeddings, attention and masks

Formula first; every symbol explained below. Natural logs unless stated.

1. Token lookup and Q/K/V projections

$$X_0 = \sqrt{d_{model}}\,E[\text{token IDs}] + PE$$
$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

X_0 – Embedding-plus-position matrix entering the Transformer.
E[token IDs] – Learned embedding vectors selected by vocabulary indices.
d_model – Token feature/embedding width.
PE – Positional encoding added to the embeddings.
X – Input token-representation matrix at the attention layer.
W_Q, W_K, W_V – Learned query, key and value projection matrices.
Q, K, V – Query, key and value matrices after projection.
Remember: The original Transformer scales token embeddings by sqrt(d_model).

2. Scores, stable softmax and value mixing

$$S = \frac{QK^T}{\sqrt{d_k}} + M, \quad A = \text{softmax}_{\text{rows}}(S), \quad O = AV$$
$$A_{ij} = \frac{e^{S_{ij} - m_i}}{\sum_k e^{S_{ik} - m_i}}, \quad m_i = \max_j S_{ij}$$
$$o_i = \sum_j A_{ij} v_j, \quad \sum_j A_{ij} = 1$$

S – Matrix of scaled query-key attention scores plus mask.
QK^T – Dot products of every query with every key.
d_k – Width of query and key vectors.
M – Additive attention-mask matrix.
A – Row-normalized attention-weight matrix.
O – Matrix of contextual outputs, A times V.
i, j – Query-row and key-position indices.
m_i – Maximum score in row i, subtracted for stability.
k inside the softmax sum – Index over all candidate key positions.
o_i – Output vector for query i.
v_j – Value vector associated with key position j.
Remember: Softmax runs across keys; mix VALUES after normalization, not keys.

3. Causal masking and dimensions

$$M_{ij} = 0 \ (j \le i), \quad M_{ij} = -\infty \ (j > i)$$
$$Q : m \times d_k, \quad K : n \times d_k, \quad V : n \times d_v$$
$$S : m \times n, \quad O : m \times d_v$$

M_ij – Mask entry for query position i and key position j.
j <= i – Current and earlier decoder input positions are allowed.
j > i – Future decoder input positions are forbidden.
-infinity – Score whose exponential becomes zero.
m, n – Counts of query positions and key/value positions.
d_k, d_v – Query/key width and value-vector width.
S: m x n – One score for each query-key pair.
O: m x d_v – One contextual value-width vector per query.
Remember: Self-attention uses one sequence; cross-attention uses decoder Q and encoder K/V.

Multi-head attention, FFN and LayerNorm

Formula first; every symbol explained below. Natural logs unless stated.

1. Several heads, then concatenate

$$\text{head}_r = \text{Attention}(XW_Q^r, XW_K^r, XW_V^r)$$
$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O$$

r – Attention-head index.
h – Total number of attention heads.
W_Q^r, W_K^r, W_V^r – Independent projections learned for head r.
head_r – Contextual output matrix produced by head r.
Concat – Join head outputs along their feature dimension.
W_O – Learned projection of concatenated head features.
MHA(X) – Multi-head attention output for input token matrix X.
Remember: For equal-width heads, d_k=d_v=d_model/h. Concatenate, not average.

2. Position-wise feed-forward network

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

x – Feature row of one token.
W_1, b_1 – First affine transformation weights and bias.
ReLU(a) – Nonlinearity that keeps positive a and replaces negatives by zero.
W_2, b_2 – Second affine transformation weights and bias.
FFN(x) – Transformed feature vector for this token.
Remember: The same FFN weights apply at each token position within one block.

3. Residual addition and feature normalization

$$y = \text{LayerNorm}(x + \text{Sublayer}(x))$$
$$\mu_u = \frac{1}{d}\sum_j u_j, \quad v_u = \frac{1}{d}\sum_j (u_j - \mu_u)^2$$
$$\text{LN}(u)_j = \gamma_j \frac{u_j - \mu_u}{\sqrt{v_u + \varepsilon_{num}}} + b_j$$

x – Original token features on the residual path.
Sublayer(x) – Attention or FFN result to add to the residual.
u – Summed feature vector x+Sublayer(x).
y – Final normalized output of this post-norm block.
d – Number of feature coordinates in one token vector.
j – Feature-coordinate index.
μ_u – Mean across this token's feature coordinates.
v_u – Mean squared deviation across those coordinates.
γ_j (gamma) – Learned gain for normalized feature j.
b_j – Learned shift for normalized feature j.
epsilon_num – Small stabilizing constant under the square root.
LN or LayerNorm – Per-token feature normalization with learned gain/shift.
Remember: The lecture uses post-norm; residual shapes must match. This is not batch normalization.

Position encoding and cross-entropy

Formula first; every symbol explained below. Natural logs unless stated.

1. Sinusoidal positional encoding

$$PE(p, 2r) = \sin\left(\frac{p}{10000^{2r/d_{model}}}\right)$$

$$PE(p, 2r+1) = \cos\left(\frac{p}{10000^{2r/d_{model}}}\right)$$

p - Token's zero-based position in the sequence.
r - Zero-based index of a sine/cosine coordinate pair.
2r, 2r+1 - Even sine coordinate and following odd cosine coordinate.
d_model - Width of the positional and embedding vectors.
10000 - Fixed base controlling the frequency range.
10000^(2r/d_model) - Shared denominator for BOTH coordinates in pair r.
sin, cos - Trigonometric functions evaluated in radians.
PE(p,j) - Encoding value at position p and coordinate j.
Remember: At position zero each pair is (0,1); interleave sine/cosine pairs.

2. Relative-position signal and projected content

$$\sin a \sin b + \cos a \cos b = \cos(a - b)$$

$$(c_i + p_i)^T (d_j + r_j) = c_i^T d_j + c_i^T r_j + p_i^T d_j + p_i^T r_j$$

a, b - Angles of two unprojected sinusoidal pairs.
cos(a-b) - Dot-product signal depending on the angular offset.
c_i, d_j - Projected content at query position i and key position j.
p_i, r_j - Projected position components of query i and key j.
T superscript - Transpose needed for the dot-product notation.
four expansion terms - Content/content, content/position and both position terms.
Remember: The full learned attention score need not depend only on relative position.

3. Vocabulary probabilities and loss

$$a_t = h_t W_{vocab} + b, \quad p_{tv} = \frac{e^{a_{tv}}}{\sum_u e^{a_{tu}}}$$

$$CE_t = -\sum_v y_{tv} \ln p_{tv} = -\ln p_{t,true}$$

$$L_{sum} = \sum_{t=1}^{N} CE_t, \quad L_{mean} = \frac{L_{sum}}{N}$$

$$\frac{\partial CE_t}{\partial a_{tv}} = p_{tv} - y_{tv}$$

h_t - Decoder feature vector at target timestep t.
W_vocab, b - Vocabulary projection weights and bias.
a_t, a_tv - Vocabulary logits; a_tv is the logit of token v.
p_tv - Predicted probability of vocabulary token v at step t.
v, u - Vocabulary indices; u is the normalization-sum index.
y_tv - One-hot target: 1 for the true token, 0 otherwise.
CE_t - Cross-entropy at target position t.
p_(t,true) - Probability assigned to the TRUE next token.
N - Number of non-padding target positions being scored.
L_sum, L_mean - Sum of token losses and their mean.
∂CE_t/∂a_tv - Per-token logit derivative, p_tv-y_tv.
Remember: Use the true token, not the argmax; averaged loss divides gradients by N.

Sequence generation and model sizes

Formula first; every symbol explained below. Natural logs unless stated.

1. Autoregressive probability and decoding

$$P(y_{1:N} \mid x) = \prod_{t=1}^{N} p(y_t \mid y_{<t}, x)$$

$$score(y_{1:N}) = \sum_{t} \ln p(y_t \mid y_{<t}, x)$$

$$y_t^{greedy} = \arg\max_{v} p(v \mid y_{<t}, x)$$

x – Available source/context, such as an encoded input sentence.
y_(1:N) – Complete target sequence of N tokens.
y_<t – Target prefix before position t.
p(y_t|y_<t,x) – Model probability of the next target token given context.
product over t – Multiply the conditional probabilities of all chosen tokens.
score(y_(1:N)) – Sum of token log probabilities; ranks sequence candidates.
v – Candidate next token in the vocabulary.
argmax_v – Return the candidate v with the largest probability.
y_t,greedy – Token chosen by greedy next-token selection.
beam width B – Number of candidate prefixes retained in beam search.
Remember: Beam search is approximate; length conventions matter for unequal-length candidates.

2. Embedding and vocabulary-head counts

$$\#embedding\ params = |V| d_{model}$$

$$\#vocab\text{-}head\ params = d_{model}|V| + |V|$$

|V| – Vocabulary size: number of distinct token entries.
d_model – Width of each token representation.
|V| d_model – Number of entries in the learned embedding table.
d_model |V| – Number of vocabulary projection weights.
final |V| term – One optional bias per vocabulary output token.
#params – Count of distinct trainable scalar parameters.
Remember: Tying vocabulary-head and embedding weights changes the distinct parameter count.

3. Common operators and overloaded symbols

ln / exp – Natural logarithm and its inverse exponential.
sum symbol (Σ) – Add all indexed terms.
product symbol (Π) – Multiply all indexed factors.
∂ – Partial derivative: sensitivity to one argument.
grad – Vector of partial derivatives over a parameter vector.
T superscript – Matrix/vector transpose, not a diffusion timestep.
I – Identity matrix; I(X;Y) instead denotes mutual information.
D – Discriminator in GANs, data dimension in MSE formulas.
σ – Standard deviation in Gaussians, sigmoid when written σ(a).
β – Diffusion noise variance; β_KL is a separate VAE loss weight.
epsilon – Random noise; epsilon_num is instead a fixed numerical stabilizer.
Remember: Definitions are local to each formula family; the same letter can have different roles.